Balancing Exploration and Exploitation in Evolutionary Algorithms

A Comparative Study of Static vs. Dynamic Mutation Parameters in the EvoMan Framework

Alfred Jose
Vrije Universiteit Amsterdam
Amsterdam, Netherlands, 2856566
a.jose@student.vu.nl

Christy Esmee Mulder
Vrije Universiteit Amsterdam
Amsterdam, Netherlands, 2754753
C.e.mulder@student.vu.nl

Daniël Wielenga
Vrije Universiteit Amsterdam
Amsterdam, Netherlands, 2742427
d.s.wielenga@student.vu.nl

Noah Wolters
Vrije Universiteit Amsterdam
Amsterdam, Netherlands, 2848625
n.j.wolters@student.vu.nl

ABSTRACT

This paper compares static versus dynamic mutation rates on the performance of evolutionary algorithms using the EvoMan framework. It was investigated how each mutation would differ in exploration and exploitation across generations. The results show that dynamic evolutionary algorithms achieve higher fitness peaks, but with slower convergence and increased variance.

KEYWORDS

Evolutionary Algorithm, Dynamic Mutation, Static Mutation

1 INTRODUCTION

This paper aims to explore how balancing exploration and exploitation works within the EvoMan framework [3]. EvoMan is a platform where players or algorithms fight bosses, useful for evaluating evolutionary algorithms. The framework's varied stages and characteristics define levels of complexity that pose challenges for algorithms to overcome.

The trade-off between exploration and exploitation was investigated by employing two different Evolutionary Algorithms (EAs): one with a static mutation parameter, one with a dynamic one. Other components of the algorithms were kept equal between both EAs. Still, since the generation of samples is a stochastic process, random variations in the data generation cannot be eliminated. Simulated annealing was used in the dynamic algorithm, it promotes early exploration, which encourages exploitation in later stages. Both EAs use the same mutation method, namely Gaussian mutation. It induces Gaussian noise into the individuals genome, here the neural network weights. The magnitude of the noise depends on two factors: the standard deviation of the noise and the current mutation rate.

To evaluate the effects of these methods and different mutation parameter selections, we simulate both EAs using the "Individual Evolution" mode. This approach results in a generalist agent trained on subsets of enemies within the EvoMan framework. For each group, we set up an independent experiment for each specific generalist agent. The question comes down to the following: The effect of static vs. dynamic mutation operators on the performance of EAs in the EvoMan framework.

2 METHODS

In this section, we talk about the experimental framework and methods used. First, we talk about balancing exploration and exploitation, after which we will discuss the selected agents and why we chose to use them. Additionally, the implemented algorithm is described. Furthermore, we will outline how and why we came to our selected parameter settings and further discuss how results will be analyzed to show the difference in performance between the different mutation operators.

2.1 Exploration vs Exploitation

Finding the balance between exploration and exploitation is important to ensure the best search behaviour in our solution space. Exploration investigates diverse areas of the search space to discover new and potentially better solutions, while exploitation refines existing high-quality solutions by searching around promising areas to find the optimal or near-optimal solutions. The performance of all EA's are heavily influenced by the dimensionality of the problem [6]. We implemented two EA's variants to look at the difference between the mutation methods. One algorithm employs a static mutation parameter, having a fixed mutation rate throughout entire runs. The second algorithm has a dynamic mutation parameter, which emphasizes on high exploration during the initial generations, gradually decreasing the mutation rate over time and encouraging exploitation in later generations.

2.2 Enemies

EvoMan has eight enemies with varying levels of difficulty along with different gimmicks between them. The enemies were divided into groups of two, it was found that the best pairs in terms of fitness levels after training were; (2,4),(2,6),(7,8) and (1,5) [2].To make them suitable for the experiment, we made two groups to train both the EA's on and test against the entire group of agents. The two enemy subsets were: group 1: enemies (1,5,8) and group 2: enemies (2,4,6). Both algorithms were trained on each group for ten runs. The mean fitness, STD σ and best fitness were recorded per run. Once training was completed, for each run, the best solution was tested on the entire group of enemies in order to assess the degree of performance against unknown enemies.

2.3 Algorithm Description

The algorithm consists of an **island-based approach**. Migrant selection is operationalized using similarity. This approach complements the question of **balancing exploration and exploitation** using EAs. Firstly, the individual with the highest fitness is being located. Secondly, another island is chosen at random (excluding the destination island), the sum of absolute differences between each individual in the source island and the destination island's best individual are being estimated. Depending on the migration size parameter n individuals with the lowest sum of absolute differences are relocated on the destination island. Migration is triggered every 5 runs, excluding the initial generation.

N-point uniform crossover was chosen as it could have an even mixture of parents increasing the potential of exploitation while still maintaining the diversity compared to single or 2-point crossovers. It works by selecting two parents through tournament selection, then n points are chosen at random within the parents genome. Between these points segments are being switched between the parents to produce offspring. This results in two new offspring with combined genetic material from both parents.

Tournament selection was implemented to determine the parents, the reason being that it maintains simplicity while also being efficient which **reduces computational costs**. The tournament size parameter determines how many individuals are being selected to participate in a tournament without replacement. The top two individuals are being selected as parents per tournament.

While tournament selection provides a state of natural elitism, and extra layer of **Elitism** was applied in the algorithm to ensure that the top 5% of individuals are preserved. This helps to ensure that top performing individuals are not lost through the stochastic nature of parent selection.

In the mutation operator **Gaussian noise** was utilized to mutate individuals. This operator was implemented to have **small, controlled mutations** which could vary from fine tuned adjustments to potentially large ones. The mutation rate, ranging from 0 to 1, determines if an individual will be mutated. This procedure is applied to all individuals. If an individual was selected to be mutated Gaussian noise was applied to each element of the individuals genome using the following formula, here sigma functioning as the standard deviation of the noise parameter with a mean of 0

allowing for negative and positive mutations.

$$\sigma = 0.1 \times \left(1 - \frac{\text{current mutation rate}}{\text{initial mutation rate}}\right)$$

The formula adjusts the noise added to the individual based on evolutionary progression. Early in the evolutionary process the difference between current and initial mutation rate is small, resulting in higher magnitudes of noise that is applied to the individuals genome. As the difference between current and initial mutation rate gets larger the noise added to the genome will become smaller.

2.4 Parameter Tuning & Setup

Parameter tuning is very important for the functioning of EAs [5]. The most important one in this case is the mutation rate. For the static mutation rate, we chose a value of 0.2. The lower this value, the more the algorithm exploits the solution rather than explores. The static mutation rate let's us observe how the algorithms perform when this parameter is consistent. In contrast, the adaptive mutation rate varies through the generations. It starts at 0.3, but lowers over time with a final mutation rate of 0.05. This helps start the gene pool to **be more explored** and then e**xploit them to their fittest versions** as the generations go on. For the crossover operation, we went with n=3, as that results in less computations as well as enough correction for enough contribution per parent [4]. Along with the mutation and crossover parameters we had to choose sizes for the island model. Three islands were created, each with 100 individuals evolving over 100 generations. The information gained changes the values in the 10 hidden neurons in the neural network architecture of the agents. We used an elitism rate of 0.05, meaning that 5 percent of the population that is strongest was kept deterministically based on their fitness. This prevents the loss of high-quality solutions during the evolution process [1]. The tournament selection parameter was set to 3, discarding one individual per tournament.

2.5 Analysis

The trained algorithms will be analysed by each training group, plotting the average/std of the mean and maximum fitness across the generations using a line-plot. The average over the 10 runs is taken of the mean and the maximum over the population in each generation. To evaluate the performance of the different algorithms we use a few statistical and visual methods. For each EA, the key metrics are considered the mean fitness and maximum fitness across all generations. We average them out over 10 independent runs to ensure reliability and significance of the results. For each group of enemies trained on, we compute the average fitness and the standard deviation of the population for each generation. That way we are able to track progression and variability between runs and in runs themselves. Along with that, we track the maximum fitness found within the population for each generation. The analysis is presented in the form of box plots that depict the gain estimated as player life - enemy life, per algorithm. The average values for each run are plotted in the center of the box, with quartiles and extremes outlining the limits of the outer cases. By comparing how far the boxes reach and how high up they get, we can compare consistency and the limits of both of the EAs, and seeing which converges faster or reaches a higher fitness in the end.

3 RESULTS

Each training group is shown in the plot and contains four lines, the mean of mean fitness and the mean of maximum fitness over the generations per algorithm. The results for the line plots are divided by the two different training groups, each containing two lines for the mean of the mean fitness and two for the mean of the maximum fitness. The values represent the averages over 10 independent runs. In group 1 (Figure 1), we see that mean of bests

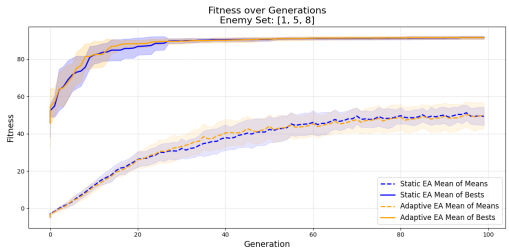

Figure 1: Group 1, Mean and Maximum Fitness over Generations per Algorithm with Standard Deviation

is comparable and is limited to about 92. For the mean of means the fitness is also very comparable between the algorithms, but the variation of the adaptive algorithm is higher. In group 2 (Figure

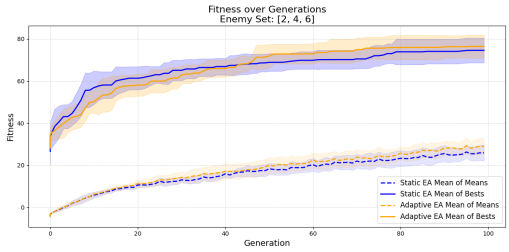

Figure 2: Group 2, Mean and Maximum Fitness over Generations per Algorithm with Standard Deviation

2, we see the mean of bests for the adaptive algorithm overtake the static algorithm and keeping it, with the highest individual point being higher as well after 100 generations for the adaptive algorithm. The difference is less notable for the mean of means (also due to the scale), but it still outperforms the static algorithm slightly. The two algorithms were tested against all the enemies. The box plots visualize the performance of the final solutions that were tested against all enemies in terms of gain. Each enemy is tested against the best solution for each of the ten independent runs, with one pair of box plots per training group. The box plot that was trained on Figure 3, shows the distribution of the gain of the algorithms tested against both of the enemies. The gain is calculated by subtracting the player health with the enemy health. As can be seen for the algorithm trained on enemies 1, 5 and 7, the average gain for both the dynamic and static algorithm is below 0, indicating a net health loss. The averages differ about 10 health, but the upper limits differ more than 20, showing limits of the dynamic version are higher than the static. Still, the difference in gain between the

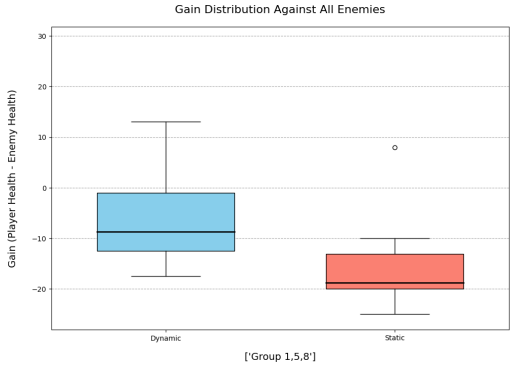

Figure 3: Trained on Enemies 1, 5, 8. Dynamic and Static

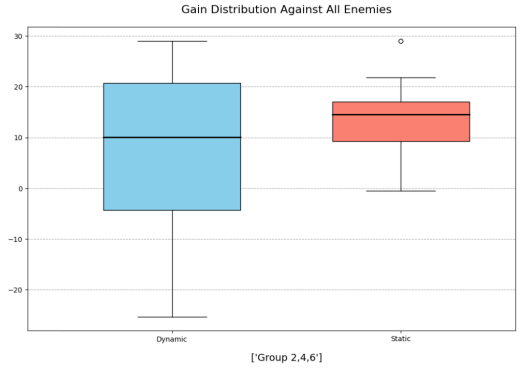

Figure 4: Trained on Enemies 2, 4, 6. Dynamic and Static

algorithms is not significant. In figure 4, we see the big difference in variance between the dynamic and static algorithms. The IQR for the dynamic reaches over around 25 gain, while the static reaches over about 7. The median for the static is also higher, but once again the upper limit for the dynamic algorithms does reach higher in Q3 and above. Still, the differences in gain are not statistically significant.

Table 1 shows the energy points of the player and the enemy for each enemy in our best solution. The best solution was able to beat five out of eight enemies.

Table 1: Energy Points of Player and Enemy for Each Enemy in the Best Solution

Enemy	Player Energy	Enemy Energy
1	100.0	0.0
2	0.0	60.0
3	26.0	0.0
4	0.0	80.0
5	83.2	0.0
6	0.0	50.0
7	91.0	0.0
8	75.4	0.0

4 DISCUSSION

Dynamic parameter adjustments, specifically in the mutation rates, impact the overall performance of the EAs. The main takeaway from the analysis is that the dynamic versions of the island-based EAs have a higher upper limit in terms of fitness. However, this comes at the cost of slightly slower convergence and greater overall variance when compared to their static counterparts, especially evident in group 2. Static EAs might be preferable in situations where computational resources or time are limited, or when predictability is necessary. Still, the findings of this study could also be due to limited impact of mutation itself, meaning that crossover operators could have driven the main progression of the algorithm. This limits reliability of the current findings. Future studies are needed in order to verify current findings.

5 CONCLUSION

To conclude, dynamic EAs show a clear advantage in achieving higher upper bounds for fitness, have a higher computational time and are more inconsistent. This shows us that although inconsistent, the dynamic EA can achieve better results and outperform the static EA while also maintaining a healthy balance of exploration and exploitation. Future work should explore different and more extended parameter values and different algorithm versions. We believe that that the dynamic mutation rate will surpass the static algorithm to more optimal solutions in the long term.

REFERENCES

[1] Chang Wook Ahn and Rudrapatna S Ramakrishna. 2003. Elitism-based compact genetic algorithms. *IEEE Transactions on Evolutionary Computation* 7, 4 (2003), 367–385.

[2] Karine da Silva Miras de Araújo and Fabrício Olivetti de França. 2016. An electronic-game framework for evaluating coevolutionary algorithms. *arXiv preprint arXiv:1604.00644* (2016).

[3] Fabricio Olivetti de Franca, Denis Fantinato, Karine Miras, AE Eiben, and Patricia A Vargas. 2019. Evoman: Game-playing competition. *arXiv preprint arXiv:1912.10445* (2019).

[4] Kenneth A De Jong and William M Spears. 1992. A formal analysis of the role of multi-point crossover in genetic algorithms. *Annals of mathematics and Artificial intelligence* 5 (1992), 1–26.

[5] Franklin Cardeñoso Fernandez and Wouter Caarls. 2018. Parameters tuning and optimization for reinforcement learning algorithms using evolutionary computing. In *2018 International Conference on Information Systems and Computer Science (INCISCOS)*. IEEE, 301–305.

[6] Mohammad Nabi Omidvar, Borhan Kazimipour, Xiaodong Li, and Xin Yao. 2016. CBCC3—A contribution-based cooperative co-evolutionary algorithm with improved exploration/exploitation balance. In *2016 IEEE congress on evolutionary computation (CEC)*. IEEE, 3541–3548.